B.Tech. BCSE497J - Project-I

PHANTOM: CAUSAL ATTRIBUTION OF RUNTIME SBOM DRIFT VIA eBPF BEHAVIORAL CONTRACTS IN KUBERNETES

Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Technology in Computer Science and Engineering

by

Register Number	Student Name and Email
23BDS0078	**HARSH RANJAN** harsh.ranjan2023@vitstudent.ac.in
23BDS0191	**JAI GOEL** jai.goel2023@vitstudent.ac.in

Under the Supervision of
Dr. THANUJA R
Associate Professor Grade 1
thanuja.r@vit.ac.in
Vellore Institute of Technology
School of Computer Science and Engineering (SCOPE)



October 2026

DECLARATION

We hereby declare that the project entitled *PHANTOM: Causal Attribution of Runtime SBOM Drift via eBPF Behavioral Contracts in Kubernetes*, submitted for the award of the degree of Bachelor of Technology in Computer Science and Engineering to Vellore Institute of Technology, is a record of project work carried out under the supervision of Dr. Thanuja R. The technical statements and results in this report are limited to the source code and records identified in the report. This work has not been submitted in whole or in part for another degree or diploma.

Place: Vellore
Date: October 2026

Harsh Ranjan Jai Goel

CERTIFICATE

This is to certify that the project entitled *PHANTOM: Causal Attribution of Runtime SBOM Drift via eBPF Behavioral Contracts in Kubernetes*, submitted by Harsh Ranjan and Jai Goel, School of Computer Science and Engineering, Vellore Institute of Technology, is a record of project work carried out under my supervision. This report describes the current research prototype and distinguishes implemented software from results that remain experimentally unverified.

Place: Vellore
Date: October 2026

Signature of the Guide
Dr. Thanuja R
Associate Professor Grade 1

ACKNOWLEDGEMENTS

We express our sincere appreciation to Dr. Sendhil Kumar K. S., Professor Grade 1 and Head of the Department, for his guidance and support. We are grateful to our project guide, Dr. Thanuja R, for her supervision and feedback, and to Vellore Institute of Technology for providing the academic environment and facilities for this Project-I work. We also thank the maintainers of the open-source projects and research tools referenced in this report.

Harsh Ranjan
Jai Goel

TABLE OF CONTENTS

DECLARATION ..... **i**

CERTIFICATE ..... **ii**

ACKNOWLEDGEMENTS ..... **iii**

LIST OF ABBREVIATIONS ..... **ix**

SYMBOLS AND NOTATIONS ..... **x**

ABSTRACT ..... **xi**

CHAPTER 1 ..... **1**

1. INTRODUCTION **1**

1.1 Background 1

1.2 Motivation 1

1.3 Scope of the Project 1

1.4 Organization 1

CHAPTER 2 ..... **2**

2. PROJECT DESCRIPTION AND GOALS **2**

2.1 Literature Review 2

2.2 Research Gap 2

2.3 Objectives 2

2.4 Problem Statement 2

2.5 Project Plan 2

CHAPTER 3 ..... **3**

3. TECHNICAL SPECIFICATION **3**

3.1 Requirements 3

3.1.1 Functional Requirements 3

3.1.2 Non-Functional Requirements 3

3.2 Feasibility Study 3

3.2.1 Technical Feasibility 3
3.2.2 Economic Feasibility 3
3.2.3 Social Feasibility 3
3.3 System Specification 3
3.3.1 Hardware Specification 3
3.3.2 Software Specification 3
CHAPTER 4 4
4. DESIGN APPROACH AND DETAILS 4
4.1 System Architecture 4
4.2 Design 4
4.2.1 Data Flow Diagram 4
4.2.2 Use Case Diagram 5
4.2.3 Class Diagram 5
4.2.4 Sequence Diagram 6
CHAPTER 5 7
5. METHODOLOGY AND TESTING 7
5.1 Module Description 7
5.2 Methodology 7
5.3 Testing 7
CHAPTER 6 8
6. PROJECT DEMONSTRATION 8
6.1 Available Demonstration Path 8
6.2 Completed-Run Evidence 8
CHAPTER 7 9
7. RESULT AND DISCUSSION 9
7.1 Experimental Setup 9
7.2 Event Presence and Latency 9

7.3 Causal Attribution 9
7.4 PCEPS Results 9
7.5 Baseline Comparison 9
7.6 Overhead and Cost 10
7.7 Threats to Validity 10
7.8 Proposed Follow-Up Evaluation 10
CHAPTER 8 11
8. CONCLUSION 11
CHAPTER 9 12
9. REFERENCES 12
APPENDIX A 14
A. ALGORITHM AND REPRODUCIBILITY CHECKLIST 14
A.1 Evidence-Gated Processing 14
A.2 Reproducibility Checklist 14

LIST OF FIGURES

Figure 4.1: Logical service architecture from repository modules 4
Figure 4.2: Data flow and evidence gates 4
Figure 4.3: Primary project use cases 5
Figure 4.4: Conceptual entity relationships 5
Figure 4.5: Simplified report-generation sequence 6

LIST OF TABLES

Table 2.1: Project work status 2
Table 3.1: Environment facts in the repository 3
Table 4.1: Main use cases 5
Table 5.1: Repository modules and status 7
Table 7.1: Run-level observed outcomes 9
Table 7.2: Injection-to-first-recorded-event latency 9

LIST OF ABBREVIATIONS

ATE	Average Treatment Effect
BDG	Behavioral Dependency Graph
BIC	Bayesian Information Criterion
CO-RE	Compile Once – Run Everywhere
eBPF	Extended Berkeley Packet Filter
ECE	Expected Calibration Error
FPR	False-Positive Rate
IMA	Integrity Measurement Architecture
KL	Kullback–Leibler Divergence
PCEPS	Predictive Compromise Exposure Probability Score (unvalidated prioritization score)
PURL	Package URL
RCA	Root-Cause Analysis
SBOM	Software Bill of Materials
SCM	Structural Causal Model

SYMBOLS AND NOTATIONS

G=(V,E)	Typed Behavioral Dependency Graph
Sigma	Behavioral token alphabet
h	Markov context
N	Number of training tokens
K_max	Maximum candidate model order
D_KL	Conditional divergence score, in nats
lambda, Delta	BDG decay parameter and time scale
q_e	Incremental edge-evidence weight
S, D, X	Candidate treatment, drift outcome, and covariates

ABSTRACT

Cloud-native applications combine software components whose behavior may change after deployment. A software bill of materials (SBOM) records component inventory, while runtime monitors observe process, file, and network activity. Neither source alone establishes which component produced an event or whether a component change caused behavioral drift. This project studies PHANTOM, a research prototype connecting SBOM-facing identity and verification interfaces with eBPF runtime collection, variable-order Markov behavioral contracts, a time-decayed Behavioral Dependency Graph (BDG), a DoWhy causal-estimation pathway, an XGBoost-based PCEPS scoring path, and evidence reporting.

The repository implements contract training with Bayesian Information Criterion (BIC) order selection, Laplace-smoothed transition probabilities, and Kullback–Leibler (KL) divergence scoring. It also contains typed graph updates, a temporal causal projection, explicit not_identifiable outcomes, a 16-feature PCEPS pipeline, and report-generation services. These are implementation capabilities; they have not all been validated together in a live deployment. The drift scorer does not implement a calibrated alert threshold. The temporal ordering is a modeling assumption and does not establish causation. PCEPS model and calibration outputs were not present in the supplied experiment artifacts.

The saved evaluation set contains nine records: six completed attacks across XZ-style backdoor and dependency-confusion scenarios, and three failed SolarWinds-style injections. Each completed run contains a recorded drift event. Injection-to-first-event latency has a mean of 15.74 seconds, median of 5.52 seconds, and linearly interpolated 95th percentile of 54.72 seconds. This is an event-presence result, not verified package-level detection accuracy. Event records lack resolved SBOM/PURL bindings and independent component-level ground truth. The evaluation also lacks valid benign controls and reproducible CPU, memory, and event-loss measurements. Therefore false-positive rates, causal attribution accuracy, predictive validity, and operational overhead cannot be established. This report documents the prototype, its evidence limits, and experiments needed to evaluate identity resolution, causal attribution, and risk prioritization under controlled conditions.

Keywords: software supply-chain security, Kubernetes, eBPF, runtime drift, SBOM, causal estimation, behavioral contracts.

CHAPTER 1

1. INTRODUCTION

1.1 Background

Cloud-native systems combine third-party packages, container images, build systems, and deployment configuration. Supply-chain incidents such as XZ Utils and SUNBURST show that a trusted distribution path can still produce behavior that warrants investigation [1]–[4]. An SBOM describes components at a point in a build or scan [10], [11], while runtime monitors observe operating-system events and may apply rules or policies [7]–[9]. Neither alone always links an event to the package responsible or establishes causation.

1.2 Motivation

A process event can be difficult to interpret when code is loaded from multiple packages or a package changes without changing its name. An inventory does not show what a deployed component did. PHANTOM studies how identity, runtime behavior, graph evidence, and explicit uncertainty can be combined without treating an association as causal proof.

1.3 Scope of the Project

The project includes SBOM verification interfaces, eBPF event collection, image/PURL-scoped contracts, drift scoring, BDG maintenance, causal-estimation eligibility, PCEPS, and evidence reports. The saved evaluation supports only event presence and first-event latency. Component attribution, calibrated prediction, false-alarm behavior, and overhead remain unestablished.

1.4 Organization

Chapter 2 describes goals and related work; Chapter 3 technical requirements; Chapter 4 design; Chapter 5 methodology and tests; Chapter 6 demonstration pathway; Chapter 7 results and discussion; Chapter 8 conclusion; Chapter 9 references.

CHAPTER 2

2. PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

Falco, Tetragon, and Tracee provide runtime events, rules, policy enforcement, or forensic capabilities depending on configuration [7]–[9]. CycloneDX and NTIA guidance address inventory and exchange [10], [11]. Syft and Cosign provide SBOM generation and signing/verification tools [20], [21], [27]. Bomfather studies build-time eBPF dependency observation and SBOM generation [29]; eBPF/IMA work studies SBOM enforcement in Linux [30]. Runtime/SBOM integration alone is therefore not PHANTOM's novelty. DoWhy is a general causal workflow [16]; applying it to runtime evidence depends on graph assumptions, treatment/outcome definitions, confounding control, and independent ground truth. Cloud incident RCA is adjacent work, but ranked explanations are not automatically intervention estimates [14], [15].

2.2 Research Gap

PHANTOM asks whether runtime observations can be mapped to trusted package identities and whether an assumption-explicit temporal causal path supports better explanations than event presence or graph ranking. Current records cannot answer this because PURL bindings and component-level labels are missing.

2.3 Objectives

1. Verify SBOM and image identity inputs.
2. Train image/PURL-scoped behavioral contracts.
3. Calculate divergence and novelty without calling them probabilities.
4. Maintain a typed, time-decayed BDG.
5. Estimate causal effects only under implemented eligibility checks; otherwise abstain.
6. Score PCEPS only when trained artifacts are available.
7. Produce evidence-based reports and evaluate without overclaiming.

2.4 Problem Statement

Given a trusted SBOM, identified image, contract, and runtime observations, record supported deviations and associate them with a component only when identity evidence supports the mapping. Causal outputs must retain assumptions and abstain when eligibility checks fail. The saved evaluation does not resolve events to PURLs or include an independent attribution oracle.

2.5 Project Plan

Table 2.1: Project work status

Phase	Evidence	Status / next action
Prototype	SBOM, eBPF, contract, graph, causal, scoring, gateway, reporting code	Implemented; selected unit paths tested
Initial evaluation	Six completed attacks; three failed injections	Event presence and latency; repair failed injections
Identity evaluation	Runtime identity ambiguous/unresolved	Collect independent process/container/PURL ground truth
Control evaluation	No valid benign raw3 controls	Run matched benign workload, update, load, restart controls
Prediction and overhead	No model evaluation or reliable resource samples	Evaluate held-out outcomes and collect telemetry

CHAPTER 3

3. TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional Requirements

Accept an SBOM, image digest and signature evidence; receive supported runtime events; build scoped contracts; calculate conditional KL and novelty; update a graph idempotently with decayed edges; check causal inputs and return an estimate or not_identifiable; score PCEPS only with model/calibration; and report missing evidence.

3.1.2 Non-Functional Requirements

The repository uses modular services and versioned JSON contracts. It should preserve provenance, handle duplicate events, expose missing fields, and avoid assigning unresolved processes to packages. The README lists Python 3.11+, Docker Compose, Terraform/Helm, and Linux CO-RE requirements. No production availability target or overhead bound is verified.

3.2 Feasibility Study

3.2.1 Technical Feasibility

Service code and tests exist, and infrastructure definitions include AWS EKS. Raw experiment records lack complete kernel, Kubernetes, runtime, image-digest and source-commit details. The architecture is prototyped; reproducible end-to-end evaluation remains future work.

3.2.2 Economic Feasibility

No cloud invoices, instance-hour totals, licensing costs, or staff-time estimates were found. No numerical cost estimate is made. A future cost study should record compute, storage, monitoring, and cluster duration.

3.2.3 Social Feasibility

Runtime telemetry may expose process and network metadata. A deployment should minimize collected fields, restrict access, and set retention limits. No production privacy assessment is available.

3.3 System Specification

3.3.1 Hardware Specification

Host type, CPU, memory, and kernel build for raw3 runs are *Not provided in the available project records*. Zero-valued metric snapshots do not establish resource usage.

3.3.2 Software Specification

Table 3.1: Environment facts in the repository

Item	Available information
Language	Python 3.11+ README requirement; exact run version missing
Orchestration	Docker Compose, Terraform/Helm; EKS definitions present
Telemetry	Linux eBPF/CO-RE requirement; collector version missing
SBOM	CycloneDX and PURL support in code
Causal	DoWhy adapter; experiment version missing
PCEPS	XGBoost/Platt code; trained experiment model absent
Revision	Experiment commit not linked to runs

CHAPTER 4

4. DESIGN APPROACH AND DETAILS

4.1 System Architecture

Figure 4.1 is a repository-based logical design, not proof that every stage ran in the experiments.

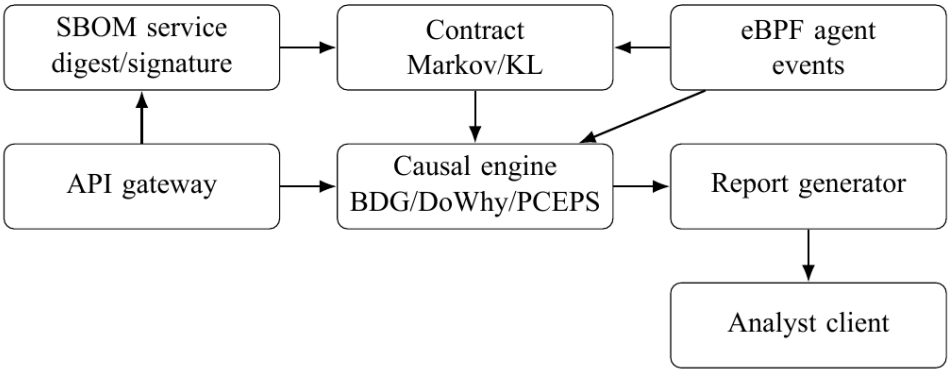


Figure 4.1: Logical service architecture from repository modules

The gateway exposes operations to the analyst client; the report generator assembles evidence. Event-to-PURL mapping is an evidence boundary: identity that cannot be resolved must remain unresolved.

4.2 Design

4.2.1 Data Flow Diagram

Figure 4.2 summarizes the evidence gates.

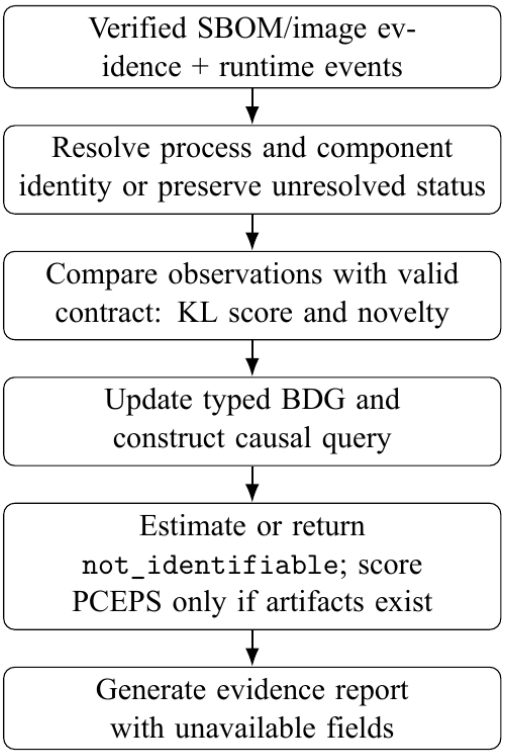


Figure 4.2: Data flow and evidence gates

1. Verify SBOM digest/signature.
2. Normalize supported events and resolve runtime identity.
3. Preserve unresolved identity where mapping evidence is missing.
4. Compare valid event windows with a contract using KL and novelty.
5. Update typed BDG evidence and temporal weights.
6. Check causal eligibility; estimate or return not_identifiable.
7. Score PCEPS only if model artifacts exist.
8. Generate a report that includes unavailable fields.

4.2.2 Use Case Diagram

The service pathway supports SBOM registration, runtime event submission, and analyst review of reports. These are code-level interactions, not a validated end-to-end user study.

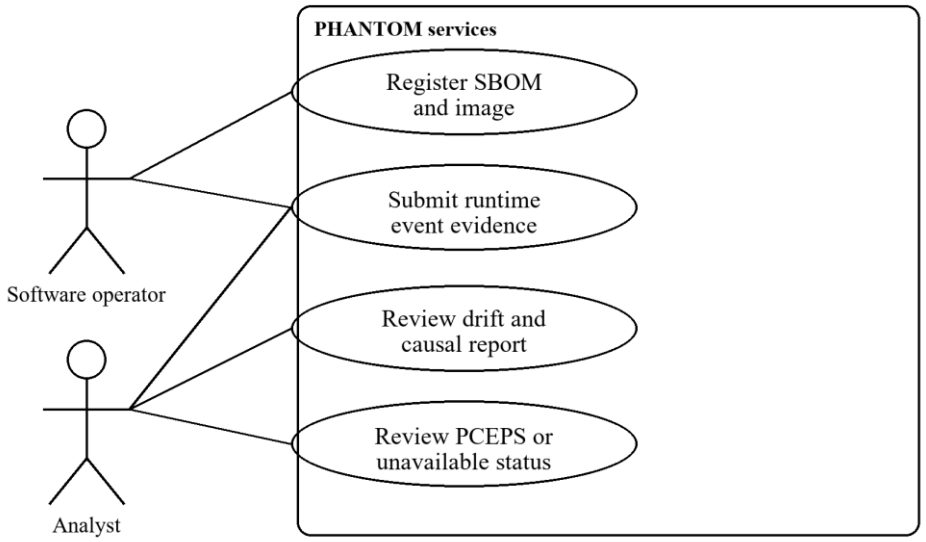


Figure 4.3: Primary project use cases

Table 4.1: Main use cases

Actor	Use case	Output
Operator	Register SBOM/image evidence	Verified/rejected artifact record
Agent	Submit event evidence	Typed event record
Analyst	Request report	Drift, identity and causal status
Analyst	Review PCEPS	Score or unavailable status

4.2.3 Class Diagram

The repository uses service/domain models rather than one monolithic class hierarchy. Conceptually, a workload/image is associated with PURLs and runtime containers/processes; events add evidence to graph relations; contracts and causal results summarize selected evidence. A process association or graph edge alone does not prove package causation.

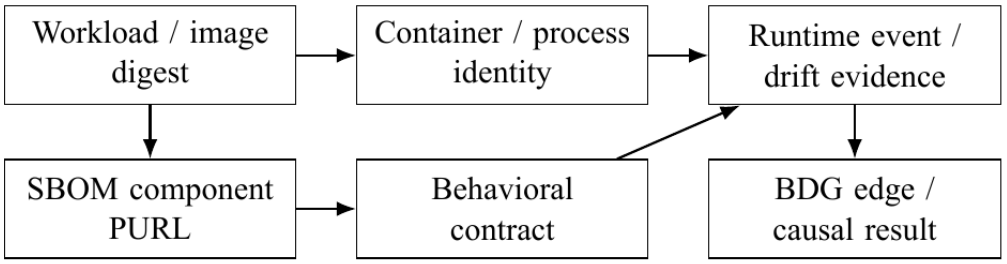


Figure 4.4: Conceptual entity relationships

4.2.4 Sequence Diagram

An analyst requests a report through the gateway. The gateway requests available graph, drift, attribution, and score records; the report generator assembles them and returns the result. If identity or scoring evidence is missing, the report should retain the unresolved or unavailable state.

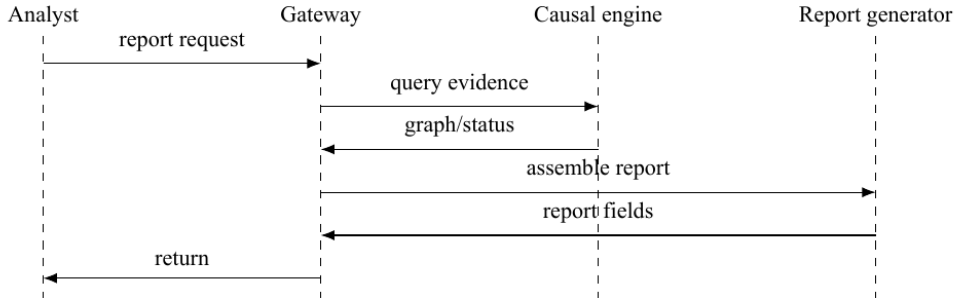


Figure 4.5: Simplified report-generation sequence

CHAPTER 5

5. METHODOLOGY AND TESTING

5.1 Module Description

Table 5.1: Repository modules and status

Module	Responsibility	Evidence status
SBOM service	Digest, metadata and signature paths	Code exists; raw3 binding unresolved
eBPF agent	Runtime event collection	Code exists; identity incomplete in raw3
Contract module	Markov training/BIC/model serialization	Implemented and unit-tested
Drift module	Conditional KL/novelty	Numeric score; no calibrated threshold
Causal engine	BDG, projection, DoWhy	Implemented paths; no attack oracle validation
PCEPS	16-feature XGBoost/Platt pathway	Training code; no evaluated artifacts
API gateway	Coordinates service operations	Code and schemas present
Report generator	Aggregates evidence	Report output does not validate cause

5.2 Methodology

For candidate order k, the Laplace-smoothed transition estimate is $\hat{p}_k(x|h) = (n_k(h,x) + 1) / (n_k(h) + |\Sigma|)$. Model selection minimizes $BIC(k) = -2 \log L_k + q_k \log N$, where $q_k = |H_k| (|\Sigma| - 1)$. Contexts are pruned when local BIC does not improve on a retained suffix. For an observed window, the weighted conditional divergence is shown below.

$$D_{\mathrm{KL}} = \sum_{h} \frac{n_h}{N} \sum_{x} \hat{P}_{\mathrm{obs}}(x \mid h) \log \frac{\hat{P}_{\mathrm{obs}}(x \mid h)}{P_{\mathrm{contract}}(x \mid h)}.$$

This is a numerical score in nats, not a calibrated anomaly probability. The scorer does not implement the proposed split-conformal alert threshold. BDG edge weights are updated as $w_e(t) = \lambda^{((t - t_{prev})/\Delta)} w_e(t_{prev}) + q_e$. The fixed semantic causal tiers are assumptions, not proof of cause.

5.3 Testing

The available local test run reported 454 tests passed and one skipped across the core library and five service suites. These tests exercise software paths, including selected graph and causal abstention behavior. They do not estimate attack-detection accuracy, prove package attribution, or establish production performance.

CHAPTER 6

6. PROJECT DEMONSTRATION

6.1 Available Demonstration Path

The repository includes service APIs, Docker Compose and infrastructure definitions, and an analyst client. A demonstrable pathway would register SBOM/image evidence, collect events, compare a contract, inspect graph and causal status, and review a generated report. The raw runs do not establish a reproducible deployed demonstration because image digests, exact versions, and package identity links are incomplete. No verified screenshot or live demonstration transcript was found, so this report does not claim a specific observed UI session.

6.2 Completed-Run Evidence

Six completed runs contain first-event timestamps within the 120-second window. Recorded events include namespace-change or privilege-transition observations, but identity remains ambiguous or missing and PURL/SBOM fields are unresolved. These traces support event-presence and latency only.

CHAPTER 7

7. RESULT AND DISCUSSION

7.1 Experimental Setup

The raw3 set has nine records: three XZ-style runs, three dependency-confusion runs, and three failed SolarWinds-style injections. Six completed attack phases are valid observations; failed injections are not false negatives. The exact kernel, Kubernetes and runtime versions, image digests, commit and experiment package versions are missing.

Table 7.1: Run-level observed outcomes

Family	Valid	Event+	Failed	Evidence status
XZ-style	3	3	0	Identity unresolved
Dependency confusion	3	3	0	Identity unresolved
SolarWinds-style	0	0	3	Injection failed
Benign control	0	–	–	No valid controls

7.2 Event Presence and Latency

All six completed runs have a recorded drift event. The event-positive fraction is 6/6 = 1.000 (100%), with a two-sided 95% Wilson score interval of [0.610, 1.000]. This is not a validated recall estimate. Latency is measured from injection timestamp to the first recorded PHANTOM event timestamp; P95 uses linear interpolation.

Table 7.2: Injection-to-first-recorded-event latency

Family	Run	Seconds
XZ-style	1	70.332
XZ-style	2	6.350
XZ-style	3	4.682
Dependency confusion	1	2.607
Dependency confusion	2	7.876
Dependency confusion	3	2.578
All	Mean	15.737
All	Median	5.516
All	Interpolated P95	54.718

The long first XZ latency's cause is not established. The unit is a scenario run, not duplicated event rows. Without valid benign controls, true negatives, precision, recall, F1, specificity, FPR, and false alarms/hour are unavailable.

7.3 Causal Attribution

An aggregate artifact reports 1.0 attribution accuracy, but raw records contain no predicted PURLs compared with oracle target PURLs. Event identities are unresolved. Graph association and correct process identification do not prove causal correctness. The temporal projection uses fixed semantic tiers and a graph constructed partly from prescribed edges. No attribution accuracy is reported.

7.4 PCEPS Results

Training, calibration, and scoring code exists. No trained model, calibration file, scored windows, held-out labels, or reproducible performance output was found. No AUC, Brier score, calibration error, or future-compromise result is reported.

7.5 Baseline Comparison

The baseline log records Falco and Trivy setup skipped, Falco alert output missing, and Isolation Forest

fit on synthetic rather than matched attack-trace frequencies. Trivy is a static vulnerability scanner. No comparative recall or superiority claim is supported.

7.6 Overhead and Cost

Saved CPU, memory and event-lag values are zero but lack raw samples and measurement provenance; they do not demonstrate zero overhead. Project cost data were not available. A future study should compare enabled/disabled monitoring under matched workloads and collect CPU, memory, application/event latency, dropped events, host versions and raw samples.

7.7 Threats to Validity

The sample covers two attack families and event presence rather than confirmed component attribution. Three injections failed, controls are absent, and package bindings are unresolved. Baseline outputs, PCEPS evaluation, overhead, and reproducibility manifest are incomplete. The causal projection depends on supplied covariates and fixed semantic ordering. These limitations prevent generalized performance or causal-correctness claims.

7.8 Proposed Follow-Up Evaluation

Freeze contracts before testing; pre-register labels; run benign package upgrades, load/configuration changes, restarts/autoscaling, both completed attacks, and repaired failed injections. Archive target PURL, image digest, process lineage, independent oracle, and PHANTOM prediction. Configure baselines for appropriate tasks on matched inputs. Split prospective PCEPS outcomes by independent run/workload. Collect resource and event-loss samples with host provenance. This is future work, not completed experimentation.

CHAPTER 8

8. CONCLUSION

PHANTOM is an implemented prototype connecting SBOM-facing services, eBPF collection, variable-order Markov contracts, a typed time-decayed graph, causal-estimation pathways, PCEPS code, and report generation. Six completed attack runs contain recorded events and measurable injection-to-first-event latency. The evidence does not establish component-level causal attribution, calibrated prediction, benign false-positive behavior, baseline performance, or overhead. A controlled follow-up with resolved identity, ground truth, controls, matched baselines and a reproducibility manifest is required before stronger claims are made.

CHAPTER 9

9. REFERENCES

[1] CISA, “Reported Supply Chain Compromise Affecting XZ Utils Data Compression Library, CVE-2024-3094,” Alert AA24-089A, Mar. 29, 2024.

[2] A. Freund, “Backdoor in upstream xz/liblzma leading to ssh server compromise,” oss-security, Mar. 29, 2024. Available: https://www.openwall.com/lists/oss-security/2024/03/29/4

[3] CISA, “Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations,” Alert AA20-352A, Dec. 2020. Available: https://www.cisa.gov/news-events/cybersecurity-advisories/aa20-352a

[4] FireEye, “Highly Evasive Attacker Leverages SolarWinds Supply Chain to Compromise Multiple Global Victims With SUNBURST Backdoor,” Dec. 2020.

[5] Apache Software Foundation, “Apache Log4j Security Vulnerabilities: CVE-2021-44228,” 2021. Available: https://logging.apache.org/log4j/2.x/security.html

[6] NIST, “CVE-2021-44228 Detail,” NVD, 2021. Available: https://nvd.nist.gov/vuln/detail/CVE-2021-44228

[7] Sysdig, “Falco: Cloud Native Runtime Security,” project documentation. Available: https://falco.org/docs/

[8] Isovalent, “Tetragon: eBPF-Based Security Observability and Runtime Enforcement,” 2023. Available: https://tetragon.io/

[9] Aqua Security, “Tracee: Linux Runtime Security and Forensics Using eBPF,” project documentation. Available: https://github.com/aquasecurity/tracee

[10] OWASP, “CycloneDX Specification, Version 1.4,” 2022. Available: https://cyclonedx.org/specification/archive/

[11] U.S. Department of Commerce, “The Minimum Elements for a Software Bill of Materials,” NTIA, 2021. Available: https://www.ntia.gov/report/2021/minimum-elements-software-bill-materials-sbom

[12] Anchore, “Anchore Enterprise: SBOM-Powered Software Supply Chain Security,” product documentation. Available: https://anchore.com/enterprise/

[13] Ox Security, “Software Supply Chain Security Platform,” product documentation. Available: https://ox.security/

[14] Y. Chen et al., “Automatic Root Cause Analysis via Large Language Models for Cloud Incidents,” in Proc. EuroSys, 2024, pp. 674–688.

[15] J. Pearl, Causality: Models, Reasoning, and Inference, 2nd ed. Cambridge University Press, 2009.

[16] A. Sharma and E. Kiciman, “DoWhy: An End-to-End Library for Causal Inference,” arXiv:2011.04216, 2020. Available: https://arxiv.org/abs/2011.04216

[17] Google, “Online Boutique: Cloud-Native Microservices Demo.” Available: https://github.com/GoogleCloudPlatform/microservices-demo

[18] DARPA, “Transparent Computing Program: Data and Resources.” Available: https://www.darpa.mil/research/programs/transparent-computing

[19] AccuKnox, “KubeArmor: Container-Aware AppArmor, SELinux, and BPF Security.” Available: https://kubearmor.io/

[20] Anchore, “Syft: Generate a Software Bill of Materials from Container Images and Filesystems.” Available: https://github.com/anchore/syft

[21] Sigstore, “Cosign: Container Signing, Verification, and Storage.” Available: https://docs.sigstore.dev/cosign/

[22] eBPF Foundation, “What is eBPF?” Available: https://ebpf.io/what-is-ebpf/

[23] A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring network structure, dynamics, and function using NetworkX,” Proc. SciPy, 2008, pp. 11–15.

[24] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” Proc. ACM SIGKDD, 2016, pp. 785–794.

[25] J. Platt, “Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods,” in Advances in Large Margin Classifiers, 1999, pp. 61–74.

[26] Package-URL Working Group, “Package URL Specification.” Available: https://github.com/package-url/purl-spec

[27] Sigstore, “Cosign Documentation: Signing Containers.” Available: https://docs.sigstore.dev/cosign/signing/signing_containers/

[28] Linux Foundation, “libbpf: BPF CO-RE Reference Guide.” Available: https://github.com/libbpf/libbpf

[29] N. Srinivasan, N. Naveen, and N. Naveen, “Bomfather: An eBPF-based Kernel-level Monitoring Framework for Accurate Identification of Unknown, Unused, and Dynamically Loaded Dependencies in Modern Software Supply Chains,” arXiv:2503.02097, 2025. Available: https://arxiv.org/abs/2503.02097

[30] A. Crawford, E. Yakubovich, and R. Szumski, “Enforcing SBOMs through the Linux kernel with eBPF and IMA,” Proc. ACM SCORED, 2023, pp. 77–78, doi: 10.1145/3605770.3625206.

APPENDIX A

A. ALGORITHM AND REPRODUCIBILITY CHECKLIST

A.1 Evidence-Gated Processing

```
verify SBOM digest and signature; reject invalid artifact
normalize supported events; resolve runtime identity
if evidence supports component PURL: associate event
else: preserve unresolved identity
if valid contract exists: compute KL and novelty
else: mark contract comparison unavailable
update typed BDG; ignore duplicate UUID; decay edge weights
if causal eligibility checks pass: estimate and refute with DoWhy
else: return not_identifiable with diagnostic
if PCEPS model and calibration exist: compute bounded score
else: mark PCEPS unavailable
generate report from available evidence
```

A.2 Reproducibility Checklist

Record the following fields for each run:

- Host type, operating system, CPU, memory, and kernel version.
- Kubernetes and container runtime versions; collector, Python, and DoWhy versions.
- Experiment commit, image and SBOM digests, and contract interval and settings.
- Target PURL, independent labels, attack and control versions, and baseline configurations.
- Raw outputs, PCEPS model and calibration digests, timestamps, event loss, resource samples, and cost records.

The current raw3 records lack several of these fields.